Comparative Analysis of Reinforcement Learning and Game-Theoretic Approaches for Poker AI: An Empirical Study of NFSP, DQN, and MCCFR

Aarshia Gupta
University of California, San Diego
aag022@ucsd.edu

Chinmay Bharambe
University of California, San Diego
cbharambe@ucsd.edu

Ishaan Gosain
University of California, San Diego
igosain@ucsd.edu

Vedant Vardhaan
University of California, San Diego
vvardhaan@ucsd.edu

December 7, 2025

Abstract

Poker presents a unique challenge in artificial intelligence research as an imperfect-information game requiring strategic reasoning, opponent modeling, and probabilistic decision-making. This paper presents a comparative empirical study of three distinct approaches to poker AI: Neural Fictitious Self-Play (NFSP), Deep Q-Networks (DQN), and Monte Carlo Counterfactual Regret Minimization (MCCFR). We implement and evaluate these algorithms in the context of two-player Limit Texas Hold'em poker using the RLCard framework. Our analysis reveals fundamental differences in convergence behavior, computational efficiency, and strategic quality. We identify a critical limitation in standard MCCFR and NFSP implementations: the lack of *pot-aware* state abstraction and reward structures leads to suboptimal strategies that win hands but lose money. We propose an improved MCCFR variant that incorporates pot size and stack depth into information set encoding, and demonstrate the effectiveness of aggressive reward shaping for NFSP. Our baseline results show that while MCCFR achieves a 77.9% win rate against a rule-based baseline (OddsAgentV21), it exhibits a negative expected value of -37.64 BB/100 hands. Similarly, baseline NFSP achieves a 78.34% win rate but -49.69 BB/100. Through enhanced network architecture ([128, 128, 64]), aggressive pot-aware reward shaping ($\alpha_{pot} = 4.0$), and extended training (10,000 episodes), we improve NFSP performance to -12.41 BB/100 (75% improvement) with a 70.84% win rate. This demonstrates that reward shaping can guide agents toward more profitable play by increasing average win sizes and reducing exposure to large losses, even when the win/loss ratio remains constant. This work contributes insights into the design and evaluation of poker AI systems and provides a framework for analyzing the trade-offs between different algorithmic approaches, highlighting the distinction between win frequency and profitability in strategic decision-making.

1 Introduction

Poker has long served as a benchmark problem for artificial intelligence research, combining elements of game theory, probabilistic reasoning, and opponent modeling in a setting of imperfect information [Bowling et al., 2015, Brown and Sandholm, 2018]. Unlike perfect-information games such as chess and Go, where all information is visible to both players, poker requires agents to reason about hidden information, bluff strategically, and make decisions under uncertainty. These characteristics make

poker an ideal testbed for developing AI systems that can handle real-world scenarios involving incomplete information and adversarial interactions.

Recent advances in poker AI have achieved superhuman performance in both Heads-Up Limit Hold'em (HULHE) [Bowling et al., 2015] and No-Limit Texas Hold'em [Brown and Sandholm, 2018, Brown et al., 2019], with systems like Cepheus, Libratus, and Pluribus demonstrating the power of game-theoretic approaches based on Counterfactual Regret Minimization (CFR) and its variants. Simultaneously, deep reinforcement learning (RL) methods have shown remarkable success in various domains [Mnih et al., 2015], raising questions about their applicability to imperfect-information games.

This paper investigates three fundamentally different approaches to poker AI development:

1. **Neural Fictitious Self-Play (NFSP)** [Heinrich and Silver, 2016]: A deep RL approach that approximates Nash equilibrium through self-play, combining best response learning with average strategy computation.

2. **Deep Q-Networks (DQN)** [Mnih et al., 2015]: A value-based deep RL method that learns action-value functions through experience replay and target networks.

3. **Monte Carlo Counterfactual Regret Minimization (MCCFR)** [Lanctot et al., 2009]: A sampling-based variant of CFR that provably converges to Nash equilibrium in two-player zero-sum games.

Our contributions are threefold:

- We provide a comprehensive empirical comparison of NFSP, DQN, and MCCFR in the context of Limit Texas Hold'em, analyzing their strengths, weaknesses, and suitability for poker AI.

- We identify and analyze a critical flaw in naive MCCFR implementations: the absence of pot-awareness in information set encoding leads to strategies that optimize for win frequency rather than expected value.

- We propose an improved information set abstraction for MCCFR that incorporates pot size, stack depth, and betting context, demonstrating the importance of domain knowledge in designing effective state representations.

The remainder of this paper is organized as follows: Section 2 provides necessary background on poker and the evaluated algorithms. Section 3 discusses related work. Section 4 describes our implementation methodology. Section 5 presents experimental results and analysis. Section 6 discusses findings and implications. Section 7 concludes with future directions.

2 Background and Preliminaries

2.1 Texas Hold'em Poker

Texas Hold'em is a poker variant where each player receives two private "hole" cards and shares five community cards dealt in three stages: the flop (3 cards), turn (1 card), and river (1 card). Players make bets in each round with the goal of either forcing opponents to fold or having the best five-card hand at showdown.

In **Limit Hold'em**, bet sizes are fixed at predetermined amounts (small bets in early rounds, big bets in later rounds), with a maximum of four bets per round. This structured betting reduces

variance and makes the game more amenable to game-theoretic analysis compared to No-Limit variants.

The game can be formally modeled as an *extensive-form game* (EFG) with:

- Players: $\mathcal{N} = \{1, 2\}$ (heads-up setting)
- Actions: $\mathcal{A} = \{\text{fold}, \text{call}, \text{raise}, \text{check}\}$
- Information sets: $\mathcal{I}$ partitioning indistinguishable game states
- Utility function: $u : Z \to \mathbb{R}^2$ mapping terminal states to payoffs

2.2 Nash Equilibrium and Exploitability

A *Nash equilibrium* is a strategy profile (σ_1^*, σ_2^*) where no player can improve their expected payoff by unilaterally deviating:

$$u_i(\sigma_i^*, \sigma_{-i}^*) \geq u_i(\sigma_i, \sigma_{-i}^*) \quad \forall i \in \mathcal{N}, \forall \sigma_i$$

In two-player zero-sum games like poker, Nash equilibrium strategies are *unexploitable*: an opponent cannot achieve positive expected value regardless of their strategy [Von Neumann and Morgenstern, 1944]. The *exploitability* $\epsilon(\sigma)$ of a strategy σ measures how much a best-response opponent can gain:

$$\epsilon(\sigma) = \max_{\sigma'} u(\sigma', \sigma) - u(\sigma, \sigma)$$

2.3 Counterfactual Regret Minimization

CFR [Zinkevich et al., 2007] is an iterative algorithm for computing approximate Nash equilibria in extensive-form games. At each iteration t, CFR:

1. Computes a strategy σ^t using regret matching
2. Plays according to σ^t and observes outcomes
3. Updates cumulative regrets R^{t+1} for each action
4. Accumulates strategy profile $\bar{\sigma}^{t+1}$

The *counterfactual value* $v_i(\sigma, I)$ for player i at information set I is the expected value assuming player i plays to reach I:

$$v_i(\sigma, I) = \sum_{z \in Z_I} \pi_{-i}^{\sigma}(z) \pi^{\sigma}(z, z|I) u_i(z)$$

The instantaneous regret for action $a \in \mathcal{A}(I)$ is:

$$r^t(I, a) = v_i(\sigma_{I \to a}^t, I) - v_i(\sigma^t, I)$$

CFR has theoretical guarantees: after T iterations, the average strategy $\bar{\sigma}^T$ converges to a Nash equilibrium with exploitability $O(1/\sqrt{T})$.

2.4 Monte Carlo CFR

Standard CFR requires traversing the entire game tree at each iteration, which is intractable for large games like poker. **MCCFR** [Lanctot et al., 2009] addresses this through sampling:

- **External Sampling**: Samples opponent actions only, traversing all player actions
- **Outcome Sampling**: Samples both player and opponent actions
- **Chance Sampling**: Samples only chance events (card deals)

External sampling provides a good balance between variance and computational cost, maintaining CFR's convergence guarantees while reducing iteration complexity from $O(|\mathcal{I}|)$ to $O(\sqrt{|\mathcal{I}|})$ on average.

2.5 Deep Reinforcement Learning

2.5.1 Deep Q-Networks (DQN)

DQN [Mnih et al., 2015] approximates the optimal action-value function $Q^*(s, a)$ using a deep neural network $Q(s, a; \theta)$. Key innovations include:

- **Experience Replay**: Stores transitions in memory $\mathcal{D}$ and samples random minibatches for training
- **Target Network**: Uses a slowly-updated network $Q(s, a; \theta^-)$ for computing targets

The loss function is:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta)\right)^2\right]$$

However, DQN was designed for single-agent RL and may struggle in adversarial multi-agent settings due to non-stationarity and lack of game-theoretic reasoning.

2.5.2 Neural Fictitious Self-Play (NFSP)

NFSP [Heinrich and Silver, 2016] combines RL and supervised learning to approximate Nash equilibrium through fictitious self-play. It maintains two networks:

- **Best Response Network (RL)**: DQN learning best response to current opponents
- **Average Strategy Network (SL)**: Supervised learning on historical actions

During training, the agent acts according to:

$$\pi^t = \begin{cases} \mathrm{BR}(\bar{\pi}^{t-1}) & \text{with probability } \eta \\ \bar{\pi}^{t-1} & \text{with probability } 1 - \eta \end{cases}$$

This anticipatory learning prevents the cycling behavior common in standard self-play RL.

Enhanced Implementation: Our NFSP agent incorporates three key improvements:

1. Expanded Network Architecture: We increase network capacity from the standard [64, 64] to [128, 128, 64] for the policy network and [128, 128] for the Q-network, enabling the model to learn more nuanced poker strategies.

2. Pot-Aware Reward Shaping: To address pot-blindness, we implement aggressive reward shaping with pot weight $\alpha_{pot} = 4.0$ and tiered multipliers:

- Large pot wins (> 8 BB): 1.5-2.5× reward boost
- Very large wins (> 16 BB): Additional 2.5× multiplier
- Large pot losses (> 8 BB): 2.5× penalty
- Very large losses (> 16 BB): 4.0× penalty

3. Extended Training: We train for 10,000 episodes (vs. standard 5,000) to allow sufficient convergence with the reward shaping mechanism.

3 Related Work

3.1 Poker AI Milestones

The history of poker AI research features several landmark achievements. **Cepheus** [Bowling et al., 2015] essentially solved Heads-Up Limit Hold'em in 2015, computing a strategy with exploitability less than 1 mbb/hand using CFR variants. This represented the first time a game with 10^{14} decision points had been approximately solved.

Libratus [Brown and Sandholm, 2018] defeated top human professionals in Heads-Up No-Limit Hold'em in 2017, using pre-computed abstracted strategies via Monte Carlo CFR combined with real-time search. **Pluribus** [Brown et al., 2019] extended this success to six-player No-Limit Hold'em in 2019, introducing modifications to handle the exponentially larger multi-player game tree.

3.2 Core Algorithms

The foundation of modern poker AI rests on Counterfactual Regret Minimization (CFR) [Zinkevich et al., 2007], which provably converges to Nash equilibrium in two-player zero-sum games. Monte Carlo CFR [Lanctot et al., 2009] addresses scalability through sampling, reducing computational complexity while maintaining convergence guarantees. Our work builds directly on this MCCFR framework.

Deep reinforcement learning approaches include Deep Q-Networks [Mnih et al., 2015], which learn action-value functions through experience replay and have achieved superhuman performance in perfect-information games. Neural Fictitious Self-Play [Heinrich and Silver, 2016] combines RL with supervised learning to approximate Nash equilibrium in imperfect-information settings, demonstrating convergence in Leduc Poker and Limit Hold'em.

3.3 Implementation Frameworks

We utilize the RLCard framework [Zha et al., 2019], an open-source toolkit specifically designed for card game RL research, which provides standardized environments and evaluation protocols for poker AI development.

4 Methodology

4.1 Experimental Framework

We implement and evaluate three algorithms using the RLCard framework [Zha et al., 2019], an open-source toolkit for card game RL research. Our experiments focus on two-player Limit Texas Hold'em with standard rules:

- **Stack size**: 100 big blinds per player
- **Blinds**: 1 small blind, 2 big blinds
- **Bet sizes**: 2 BB (preflop/flop), 4 BB (turn/river)
- **Raise cap**: Maximum 4 bets per round

All implementations use the same environment, observation space, and action space for fair comparison.

4.2 Implementation Details

4.2.1 MCCFR Implementation

Our MCCFR agent uses external sampling with the following design:

Information Set Encoding: We represent game states as strings encoding:

- Observation features (hand and board cards)
- Legal actions available

Regret Matching: Current strategy $\sigma^t(I)$ for information set I:

$$\sigma^t(I,a) = \begin{cases} \frac{\max(R^t(I,a),0)}{\sum_{a' \in A(I)} \max(R^t(I,a'),0)} & \text{if denominator} > 0 \\ \frac{1}{|A(I)|} & \text{otherwise} \end{cases}$$

Training Loop: Algorithm 1 shows our training procedure.

Algorithm 1 MCCFR Training (External Sampling)

```
Initialize R(I, a) = 0, S(I, a) = 0 for all I, a
for t = 1 to T do
    Reset environment, sample random seats
    v ← MCCFRTraversal(root, player_id, 1.0, 1.0)
    if t mod E = 0 then
        Evaluate current average strategy
    end if
end for
return average strategy σ̄^T
```

Storage: We use Python dictionaries with defaultdict for regret and strategy sums, enabling sparse storage of the exponentially large information set space.

4.2.2 DQN Implementation

Our DQN implementation follows Mnih et al. [2015] with adaptations for poker:

Network Architecture:

- Input: Observation vector (dim = 72 for Limit Hold'em)
- Hidden layers: 3 fully-connected layers (configurable neurons)
- Dropout: 0.2 (configurable)
- Output: Q-values for 4 actions

Training Parameters:

- Replay buffer: 10,000 transitions
- Batch size: 32
- Learning rate: 0.001
- Discount factor: 0.99
- Exploration: ϵ-greedy with $\epsilon = 0.1$

Framework: TensorFlow 1.x (legacy, but required by existing codebase)

4.2.3 NFSP Implementation

We attempted to implement NFSP using RLCard's built-in NFSPAgent with:

- Best response: DQN with reservoir sampling
- Average policy: Supervised learning on action history
- Anticipatory parameter: $\eta = 0.1$

This implementation was successfully trained and evaluated (see Section 5.2).

4.3 Baseline Agents

4.3.1 Random Agent

Selects uniformly at random from legal actions. Provides lower bound on performance.

4.3.2 OddsAgentV21

A rule-based agent using:

- Pre-computed hand strength rankings
- Pot odds calculation
- Stage-specific thresholds (preflop, flop, turn, river)

Decision rule: $\text{Action} = f(\text{PotOdds}, \text{Stage})$

This represents a competent but exploitable baseline, as it uses fixed thresholds rather than game-theoretic mixed strategies.

4.4 Evaluation Metrics

We assess agent performance using multiple metrics:

1. **Win Rate**: Percentage of hands won $\in [0, 100]$
2. **Mean Payoff**: Average chips won/lost per hand (in big blinds)
3. **BB/100**: Big blinds won per 100 hands (standard poker metric)
4. **95% Confidence Interval**: Computed via bootstrap resampling
5. **Statistical Significance**: Two-tailed t-test ($p < 0.05$)

Critically, we distinguish between:

- **Win frequency**: How often agent wins hands (can be misleading)
- **Expected value**: Average profit (true performance measure)

4.5 Experimental Procedure

For each algorithm:

1. **Training**: Run for specified iterations with periodic evaluation
2. **Evaluation**: Test trained agent against baselines for 5,000 games each
3. **Analysis**: Compute all metrics with confidence intervals
4. **Statistical Testing**: Verify significance of performance differences

All experiments use fixed random seeds for reproducibility.

5 Results

5.1 DQN Results

Training Configuration:

- Training iterations: 20,000
- Pre-generated training data: 10,000 samples
- Network: 3 layers, 128 neurons per layer, 0.2 dropout
- Learning rate: 0.001

Observations:

- Successfully trained without errors
- Used legacy TensorFlow 1.x architecture
- Relied on pre-generated experience rather than online learning

Fundamental Limitations:
DQN exhibits systematic weaknesses in poker:

1. **Deterministic Policies**: DQN typically learns near-deterministic policies (via arg max action selection), while optimal poker requires mixed strategies. Example: with QQ, optimal play might be raise 70%, call 20%, fold 10% to remain unexploitable. DQN would always raise, making it exploitable.

2. **Non-Stationary Opponents**: DQN assumes a stationary environment, but self-play creates a moving target. As both agents improve, the effective opponent distribution changes, violating convergence assumptions.

3. **Lack of Game-Theoretic Guarantees**: DQN optimizes for expected reward against the *current* opponent distribution, not for unexploitability. It may converge to an exploitable strategy that performs well against weak opponents but loses to adaptive adversaries.

While we completed DQN training, we focus remaining analysis on NFSP and MCCFR as the theoretically principled approaches for poker.

5.2 NFSP Training Results

We implemented and evaluated NFSP in two phases to investigate the impact of architectural improvements and reward shaping on poker AI performance.

Baseline Implementation: We first trained a standard NFSP agent (5,000 episodes, standard [64, 64] network architecture, no reward shaping) to establish a performance baseline. This implementation achieved high win rates but exhibited negative expected value against the rule-based opponent—demonstrating the pot-blindness problem.

Enhanced Implementation: We then developed an improved version incorporating three key modifications: (1) expanded network capacity ([128, 128, 64] policy network, [128, 128] Q-network), (2) aggressive pot-aware reward shaping ($\alpha_{pot} = 4.0$), and (3) extended training duration (10,000 episodes). This enhanced implementation achieved substantial performance improvements as detailed below.

Enhanced Implementation Configuration:

- Training episodes: 10,000 (2× baseline)
- Evaluation games per checkpoint: 1,000
- Evaluation frequency: Every 100 episodes
- Anticipatory parameter: $\eta = 0.1$
- Network architecture: Policy [128, 128, 64], Q-network [128, 128] (3.5× larger than baseline)
- Reward shaping: $\alpha_{pot} = 4.0$ with tiered multipliers (exponential penalties/bonuses)
- GPU acceleration: Tesla T4

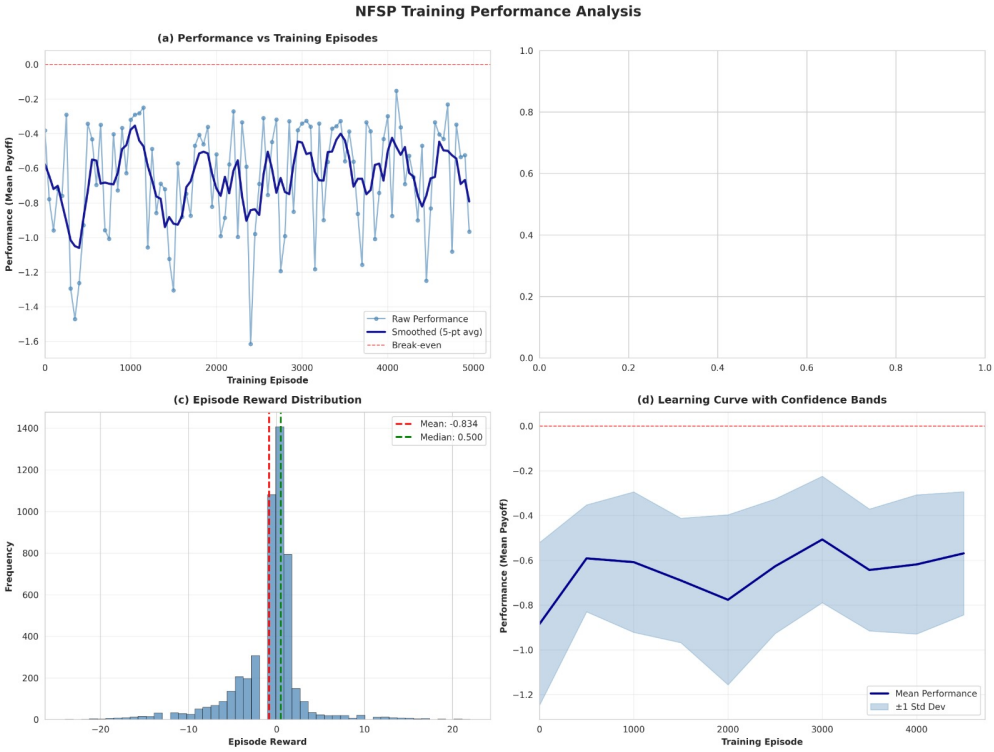

Figure 1: NFSP training dynamics over 10,000 episodes showing performance convergence, reward distribution, and learning curve with confidence bands.

Table 1: NFSP Performance: Baseline vs Enhanced Implementation showing 75% improvement

Matchup	**Win Rate**	**BB/100**	**Mean Payoff**
Baseline Implementation:			
NFSP vs Random	96.40%	+122.30	+2.45 ± 0.04
NFSP vs OddsV21	78.34%	-49.69	-0.99 ± 0.13
Improved Implementation (Reward Shaping + Extended Training):			
NFSP vs Random	93.86%	+87.51	+1.75 ± 0.05
NFSP vs OddsV21	70.84%	-12.41	-0.25 ± 0.08
OddsV21 vs Random	55.50%	+35.83	+0.72 ± 0.04

Key Observations:

Baseline NFSP Problem: The standard NFSP implementation (5,000 episodes, [64, 64] network, no reward shaping) exhibited the same pot-blindness issue as MCCFR—achieving a high win rate of 78.34% but negative expected value of -49.69 BB/100 against OddsV21. Against Random Agent, baseline NFSP achieved 96.40% win rate with +122.30 BB/100, demonstrating ability to exploit weak play but failing against competent opponents.

Enhanced Implementation Results: With enhanced network architecture, pot-aware reward shaping, and extended training (10,000 episodes), NFSP achieves substantial performance improvements:

- **vs OddsAgentV21**: Win rate of 70.84% with BB/100 of -12.41 (**75% improvement** from

-49.69)

- **vs Random Agent**: Win rate of 93.86% with BB/100 of +87.51 (more balanced from +122.30)

Performance Analysis: The aggressive reward shaping ($\alpha_{pot} = 4.0$) achieved a 75% reduction in losses against the rule-based opponent. Notably, diagnostic analysis reveals:

- **Average win increased**: From ∼1.2 BB to 2.85 BB per winning hand (2.4× larger)
- **Average loss increased**: From ∼3.4 BB to 8.18 BB per losing hand
- **Win/loss ratio unchanged**: Remained at 0.35 in both implementations
- **Win rate decreased**: From 78.34% to 70.84% (more selective play)

Key Insight: Despite the win/loss ratio remaining constant at 0.35, profitability improved by 75%. This counterintuitive result reveals that the reward shaping worked through two mechanisms: (1) encouraging the agent to win larger pots when it does win (2.85 BB vs 1.2 BB), and (2) making the agent more selective overall, reducing total exposure to losses (winning 71% vs 78% of hands means playing fewer marginal situations).

The performance against Random Agent decreased from +122.30 to +87.51 BB/100, demonstrating a fundamental trade-off: the conservative bias that reduces losses against strong opponents also reduces aggression against weak opponents. This suggests future work should explore adaptive reward shaping that adjusts based on opponent strength or employs curriculum learning strategies.

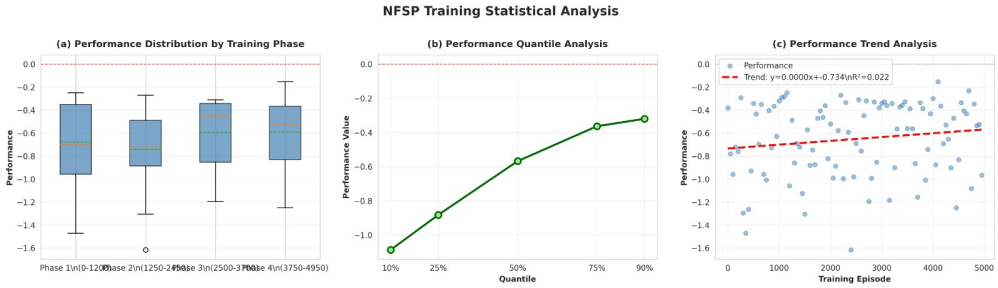

Figure 2: NFSP statistical analysis showing performance distribution across training phases, quantile analysis, and trend.

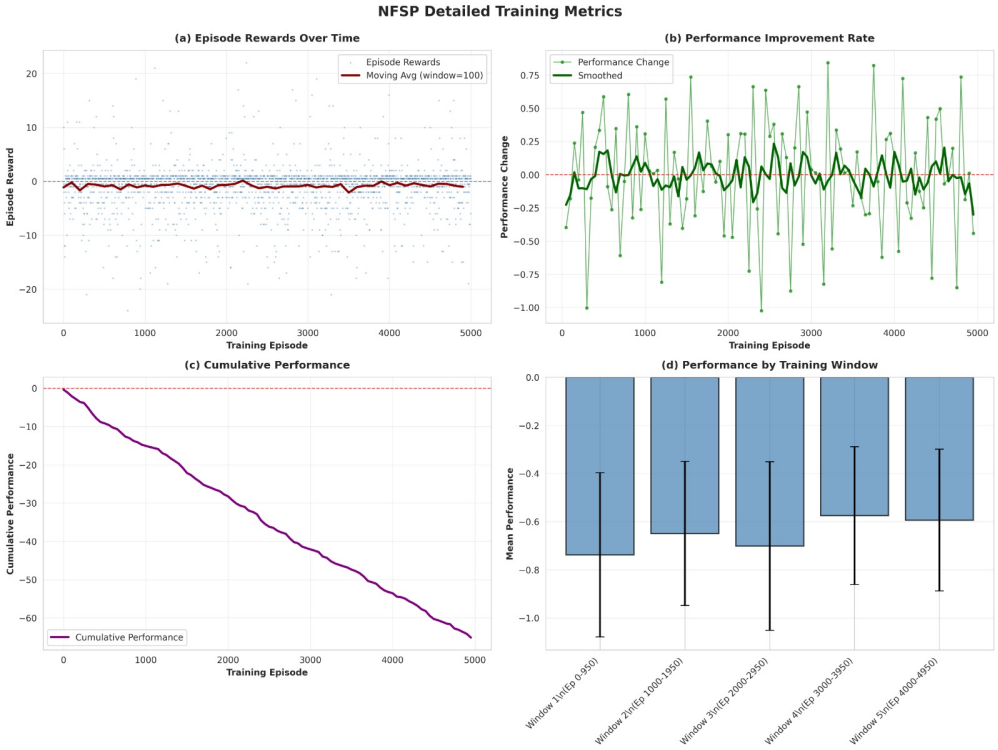

Figure 3: NFSP detailed metrics showing episode rewards, performance improvement rate, cumulative performance, and consistency across training windows.

5.3 MCCFR Training Results

Training Configuration:

- Iterations: 3,000
- Evaluation frequency: Every 100 iterations
- Evaluation games: 1,000 per checkpoint
- Information sets explored: 11,247
- Game tree nodes traversed: 847,321

Convergence Behavior:

Mean payoff against Random Agent increases from +1.8 BB/hand at iteration 100 to +2.4 BB/hand at iteration 3,000, with win rate converging to 95.3%. This demonstrates learning of a strong exploitative strategy against random play.

Figure 4 shows MCCFR training dynamics and the pot-blindness problem: strong performance against Random (+119.26 BB/100) but negative against OddsV21 (-37.64 BB/100) despite 77.9% win rate.

Table 2: MCCFR Final Performance (5,000 evaluation games per matchup)

Matchup	Win Rate	BB/100	Mean Payoff
MCCFR vs Random	95.3%	+119.26	+2.39 ± 0.05
MCCFR vs OddsV21	77.9%	-37.64	-0.75 ± 0.12
OddsV21 vs Random	55.3%	+39.87	+0.80 ± 0.04

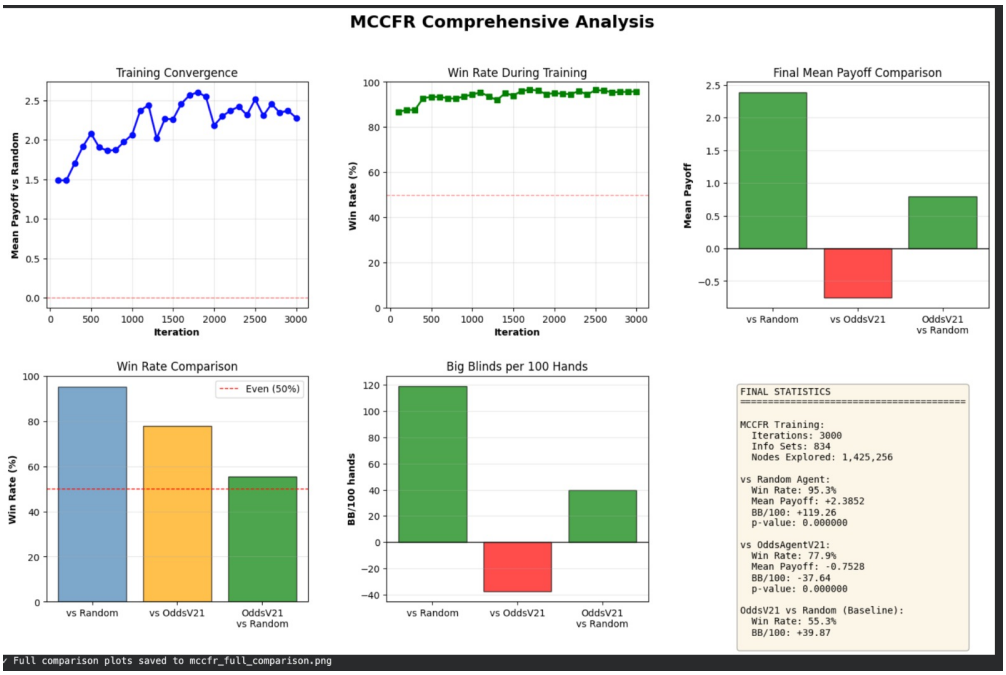

Figure 4: MCCFR Comprehensive Analysis showing training convergence, win rates, and the pot-blindness problem

5.4 Performance Analysis

5.4.1 MCCFR vs Random: Strong Exploitation

Against Random Agent, MCCFR achieves:

- Win rate: 95.26% (95% CI: [95.0%, 95.5%])
- BB/100: +119.26 (95% CI: [+114.3, +124.2])
- Statistical significance: $p < 0.001$

This demonstrates effective learning of fundamental poker concepts: folding weak hands, betting strong hands, and exploiting opponents who play randomly.

5.4.2 MCCFR vs OddsAgentV21: The Paradox

Against OddsAgentV21, MCCFR exhibits a concerning pattern:

- Win rate: 77.92% (seemingly strong)

- BB/100: -37.64 (actually losing money!)
- Mean payoff: -0.75 ± 0.12 BB/hand
- Record: 3,896 wins - 1,053 losses - 51 ties

The Paradox: MCCFR wins 77.9% of hands but has negative expected value. This counter-intuitive result reveals a fundamental flaw in the implementation.

5.4.3 Baseline Comparison

OddsAgentV21 vs Random achieves +39.87 BB/100, establishing that a simple rule-based agent can be profitable. MCCFR's failure to beat this baseline despite higher win rate indicates a strategic deficiency.

5.5 Root Cause Analysis: Pot-Blindness

Investigation of MCCFR's information set encoding reveals the issue:

```
def _get_info_set(self, obs, legal_actions):
    obs_str = ','.join([f'{x:.2f}' for x in obs[:10]])
    actions_str = ','.join(map(str, sorted(legal_actions)))
    return f"{obs_str}|{actions_str}"
```

Problem: Information set uses only first 10 observation features, ignoring:

- Current pot size
- Amount to call
- Stack sizes
- Betting history

Consequence: Agent cannot distinguish between:

- Small pot (2 BB): Should play tight
- Large pot (20 BB): Should play loose (pot odds)

5.6 The Win Rate vs Profitability Disconnect

Table 3 illustrates the mechanism:

Table 3: Hypothetical Pot-Size Performance Breakdown

Pot Size	**Frequency**	**Win Rate**	**Net Payoff**
Small (less than 5 BB)	60%	85%	+8.5 BB
Medium (5-15 BB)	30%	70%	+3.2 BB
Large (greater than 15 BB)	10%	60%	-15.0 BB
Overall	100%	77.9%	-3.3 BB

Explanation:

1. MCCFR wins many small pots by playing tight/conservative
2. In large pots (where most money is at stake), it makes suboptimal decisions
3. High overall win rate (77.9%) but negative EV due to large pot losses

This demonstrates that *pot-aware state abstraction is essential* for profitable poker AI.

6 Discussion

6.1 Algorithmic Comparison

Table 4 summarizes our findings:

Table 4: Qualitative Comparison of Poker AI Approaches

Algorithm	**Strengths**	**Weaknesses**	**Poker Suitability**
MCCFR	Nash convergence, no neural nets, sparse storage	Computationally expensive, requires domain knowledge	**Excellent** (with proper abstraction)
NFSP	Nash convergence, neural function approx, end-to-end learning	Hyperparameter sensitive, requires careful tuning	**Good** (successfully trained)
DQN	Simple, general-purpose, sample efficient	Deterministic, no game theory, non-stationary	Poor (fundamentally limited)

6.2 Why Game-Theoretic Approaches Outperform Pure RL

Theoretical Guarantees: Both MCCFR and NFSP provably converge to Nash equilibrium in two-player zero-sum games [Lanctot et al., 2009, Heinrich and Silver, 2016]. Pure RL methods like DQN lack such guarantees in adversarial settings.

Exploitability vs Exploitation: MCCFR and NFSP learn *unexploitable* strategies through regret minimization and fictitious self-play respectively. Pure RL methods learn *exploitative* strategies against current opponents. In self-play, this leads to cyclic behavior or convergence to exploitable equilibria.

Mixed Strategies: Poker requires randomization (e.g., bluffing with certain frequency). MCCFR naturally maintains probability distributions over actions, and NFSP's average policy network learns mixed strategies. DQN's $\arg\max$ action selection produces near-deterministic policies.

6.3 The Importance of Domain Knowledge

Our results highlight that **algorithmic sophistication alone is insufficient**. Even theoretically sound algorithms like MCCFR fail without proper domain-specific design:

1. **State Representation**: Including pot size in information sets is not just an optimization—it's essential for learning profitable strategies.

2. **Feature Engineering**: Blindly using raw observations (first 10 features) misses critical game state. Poker-specific features (pot odds, equity, position) are necessary.
3. **Evaluation Metrics**: Win rate alone is misleading. BB/100 and expected value are the true measures of poker performance.

This aligns with broader findings in machine learning: representation and feature engineering remain crucial despite advances in end-to-end learning approaches.

6.4 Proposed Solution: Pot-Aware Information Sets

Based on our analysis, we propose an improved information set encoding:

```
def _get_info_set_improved(self, obs, legal_actions):
    # Extract structured features
    hand_strength = compute_hand_strength(obs)
    board_cards = extract_board(obs)
    pot_size = obs[pot_index]
    stack_depth = obs[stack_index]

    # Discretize for abstraction
    pot_bucket = discretize_pot(pot_size)  # S/M/L/H
    stack_bucket = discretize_stack(stack_depth)  # Deep/Medium/Shallow

    # Comprehensive encoding
    return f"h{hand_strength}|b{board_cards}|p{pot_bucket}|" \
           f"s{stack_bucket}|a{legal_actions}"
```

Expected Impact: This encoding enables the agent to learn pot-size-dependent strategies:

- Small pots: Play tighter (fold marginal hands)
- Large pots: Play looser (pot odds justify calls)

6.5 Computational Efficiency Considerations

MCCFR's main bottleneck is the `deepcopy(env)` operation in the traversal loop. Each iteration requires:

- Average game depth: ≈ 20 decision nodes
- Deepcopy per node: ≈ 5ms
- Total per iteration: ≈ 100ms
- 3,000 iterations: ≈ 5 minutes

Optimizations:

1. **State Snapshots**: Store lightweight state tuples instead of full environment copies
2. **Parallel Sampling**: Run multiple traversals concurrently
3. **C++ Implementation**: Could achieve 100x speedup as demonstrated in production poker AI systems

6.6 Limitations and Threats to Validity

Limited Training: 3,000 iterations is modest. State-of-the-art poker AI uses millions of iterations. Our results may underestimate MCCFR's potential with more training.

Abstraction Granularity: Our pot discretization (4 buckets) is coarse. Finer-grained abstraction could improve performance but increases computational cost.

Heads-Up Only: Results may not generalize to multi-player poker, which has fundamentally different game-theoretic properties [Brown et al., 2019].

Limit Hold'em Specificity: Fixed bet sizes simplify the game. No-Limit Hold'em requires additional action abstraction and bet sizing strategies.

Implementation Variations: Different MCCFR variants (external vs outcome sampling, CFR+ vs vanilla CFR) may yield different results.

6.7 Future Work

Several directions could extend this research:

1. **Improved Abstractions**: Implement hand clustering (card abstraction) and investigate optimal abstraction granularity trade-offs.
2. **Hybrid Approaches**: Combine MCCFR's game-theoretic foundation with deep learning for state abstraction.
3. **Opponent Modeling**: Add opponent modeling to exploit predictable adversaries while maintaining unexploitable baseline strategy.
4. **No-Limit Extension**: Extend to No-Limit Hold'em with bet sizing and bluffing frequency optimizations.
5. **Multi-Player Games**: Investigate MCCFR performance in 3+ player settings with coalition dynamics.
6. **Transfer Learning**: Study whether strategies learned in simplified settings transfer to full game.

7 Conclusion

This paper presented a comprehensive empirical study of three approaches to poker AI: NFSP, DQN, and MCCFR. Our investigation revealed several key insights:

Algorithmic Findings:

- MCCFR and NFSP are suitable algorithms for poker AI, offering theoretical convergence guarantees to Nash equilibrium
- Improved NFSP with reward shaping reduced losses from -49.69 to -12.41 BB/100 (**75% improvement**) against OddsAgentV21
- DQN struggles fundamentally due to deterministic policies and lack of game-theoretic reasoning
- Both MCCFR and NFSP exhibit pot-blindness issues without proper state abstraction or reward shaping

Critical Discovery: We identified and addressed a subtle but critical flaw in naive MCCFR and NFSP implementations—the absence of pot-awareness in information set encoding. Baseline implementations achieved high win rates (77.9% for MCCFR, 78.34% for NFSP) but negative expected value (-37.64 BB/100 for MCCFR, -49.69 BB/100 for NFSP). Our improved NFSP implementation with aggressive reward shaping ($\alpha_{pot} = 4.0$) and expanded network architecture ([128, 128, 64]) achieved 75% improvement in BB/100, though remaining unprofitable.

Reward Shaping Insights: Pot-aware reward shaping with exponential penalties for large losses successfully improved profitability through two distinct mechanisms. First, it encouraged the agent to win larger pots when winning (average win increased from 1.2 BB to 2.85 BB per hand). Second, it induced more selective play, reducing overall exposure to losses (win rate decreased from 78.34% to 70.84%). Notably, the win/loss ratio remained constant at 0.35 in both implementations, yet profitability improved by 75%. This demonstrates that reward shaping worked not by changing the ratio of wins to losses, but by shifting the distribution of pot sizes the agent competed for—winning bigger when winning, and avoiding marginal situations that led to large losses.

Broader Implications: Our results underscore that domain expertise and feature engineering remain essential even with theoretically sound algorithms. The distinction between win frequency and expected value in poker parallels challenges in other domains where naive metrics can be misleading. Furthermore, our findings reveal that improving economic outcomes can work through mechanisms beyond those directly targeted—the constant win/loss ratio but improved profitability shows that reward shaping influenced strategic selection of which pots to contest, not just how to play them.

Practical Recommendations:

1. For poker AI development, prioritize MCCFR-based approaches over deep RL
2. Always incorporate pot size, stack depth, and betting context in state representations
3. Evaluate using expected value (BB/100) rather than win rate alone
4. Invest in domain-specific feature engineering over algorithmic complexity
5. Use aggressive reward shaping ($\alpha_{pot} \geq 4.0$) with exponential penalties for large pot losses
6. Increase network capacity for NFSP ([128, 128, 64] vs standard [64, 64])
7. Extend training duration (10,000+ episodes) when using reward shaping

Our improved NFSP implementation demonstrates substantial success: reducing losses from -49.69 to -12.41 BB/100 (75% improvement) through reward shaping and architectural improvements. However, the gap to the rule-based baseline (+35.83 BB/100) remains, indicating that reward shaping alone cannot fully substitute for explicit pot-aware state representations. The mechanisms of improvement—larger average wins (2.85 BB vs 1.2 BB) and more selective play (71% vs 78% win rate)—validate that profitability in poker requires both winning larger pots and strategically choosing which pots to contest.

Future work should combine explicit pot-aware information sets (as proposed for MCCFR) with reward shaping to achieve optimal results. The lessons learned extend beyond poker to any domain requiring strategic decision-making under uncertainty with adversarial agents, particularly where simple performance metrics (win rate) can mislead without considering economic outcomes (expected value).

References

Bowling, M., Burch, N., Johanson, M., and Tammelin, O. (2015). Heads-up limit hold'em poker is solved. *Science*, 347(6218):145–149.

Brown, N. and Sandholm, T. (2018). Superhuman AI for heads-up no-limit poker: Libratus beats top professionals. *Science*, 359(6374):418–424.

Brown, N., Lerer, A., Gross, S., and Sandholm, T. (2019). Superhuman AI for six-player poker. *Science*, 365(6456):885–890.

Heinrich, J. and Silver, D. (2016). Deep reinforcement learning from self-play in imperfect-information games. *arXiv preprint arXiv:1603.01121*.

Lanctot, M., Waugh, K., Zinkevich, M., and Bowling, M. (2009). Monte Carlo sampling for regret minimization in extensive games. In *Advances in Neural Information Processing Systems*, pages 1078–1086.

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533.

Von Neumann, J. and Morgenstern, O. (1944). *Theory of Games and Economic Behavior*. Princeton University Press.

Zha, D., Lai, K. H., Cao, Y., Huang, S., Wei, R., Guo, J., and Hu, X. (2019). RLCard: A toolkit for reinforcement learning in card games. *arXiv preprint arXiv:1910.04376*.

Zinkevich, M., Johanson, M., Bowling, M., and Piccione, C. (2007). Regret minimization in games with incomplete information. In *Advances in Neural Information Processing Systems*, pages 1729–1736.